

DESIGN DOCUMENT

Warehouse-Backed Grocery Self-Checkout & Restocking System

1. System Overview

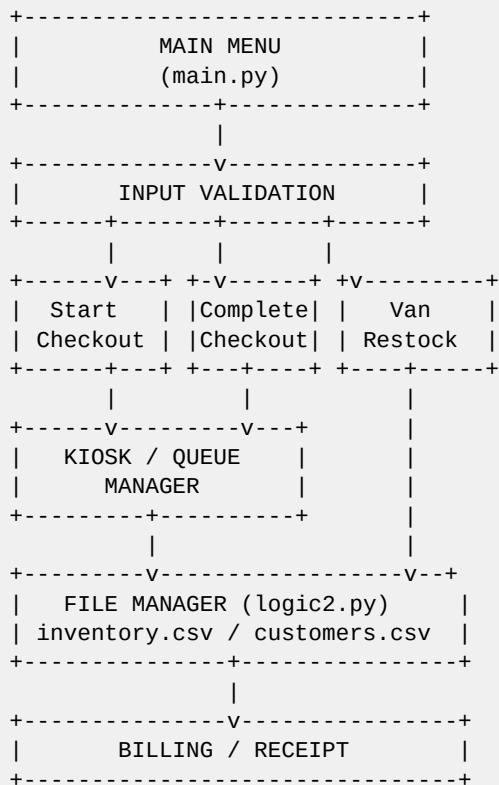
The system is a modular, console-based Python application that manages self-checkout kiosks and warehouse-style van restocking for a grocery store. All stock information is persisted in an external CSV file, which acts as the single source of truth for inventory across sessions. A second CSV file stores how many purchases each customer has completed, and the loyalty tier is worked out from it automatically.

Code structure: the application is split across three code files. `logic1.py` contains the business rules and calculations (checkout, restock, tiers, discounts, billing, and screen output). `logic2.py` contains every data structure and the functions that read, save and change them. `main.py` owns the session state and the top-level menu loop, calling into both. All three files, plus `inventory.csv`, must be in the same folder, and the application is started by running `python main.py`. A separate `test_logic.py` holds the automated tests for the PRD's test cases and edge cases.

2. Design Goals

- Keep the CSV inventory file as the single, always-accurate source of stock data.
- Handle limited kiosks gracefully by queueing customers when all kiosks are busy.
- Make the queue genuinely testable in a single terminal by separating arriving at a kiosk from finishing the checkout.
- Work out the loyalty tier automatically from stored visit history, so it cannot be typed in wrongly.
- Let van deliveries restock items without corrupting inventory data.
- Flag low-stock items automatically.
- Keep file reading and writing in a small set of functions so the rest of the system does not depend on the file format.
- Keep business rules (`logic1.py`) separate from data structures and storage (`logic2.py`).

3. System Architecture



The menu is `main.py`. Start Checkout, Complete Checkout, Van Restock, input validation and billing are in `logic1.py`. The File Manager, and the kiosk, queue, stock and cart operations they call, are in `logic2.py`.

4. Module Design

4.1 File Structure

File	Responsibility
<code>main.py</code>	Entry point. Creates the inventory, customers, kiosks, active-sessions and queue data for the session, runs the menu loop, and calls functions in <code>logic1.py</code> and <code>logic2.py</code> .
<code>logic1.py</code>	Rules and calculations: tier calculation, discount rounding, billing, start and complete checkout, queue handling, van restock, input prompts, and all printing. It never changes a data structure directly; it calls <code>logic2.py</code> .
<code>logic2.py</code>	Data structures: the constants, loading and saving <code>inventory.csv</code> and <code>customers.csv</code> , and the functions that change stock, customer visits, kiosks, the queue and the cart.
<code>inventory.csv</code>	Persistent stock data: <code>item_id</code> , <code>name</code> , <code>quantity</code> , <code>price</code> .
<code>customers.csv</code>	Persistent customer history: <code>customer_id</code> , <code>visits</code> . Created automatically on first save.
<code>test_logic.py</code>	Automated tests mapped to the PRD's test cases and edge cases.

Run the application with: `python main.py`

4.2 Modules (by function)

Module	File	Purpose
Main Menu	main.py, logic1.py	Shows kiosk status, queue and inventory, then offers Start Checkout, Complete Checkout, Process Van Delivery, View Inventory and Exit until Exit is chosen.
Start Checkout	logic1.py	Reads the Customer ID, works out the visit number and tier, then claims a free kiosk or joins the queue. Does not read a cart.
Complete Checkout	logic1.py	Lets the user pick a busy kiosk, reads that customer's cart, validates each line, deducts stock, bills with the tier discount, records the visit, and releases the kiosk.
Kiosk / Queue Manager	logic1.py, logic2.py	Finds the lowest-numbered free kiosk, and when a kiosk is released, assigns it to the customer at the front of the queue.
Van Restock	logic1.py	Reads a delivery manifest, adds the delivered quantities to stock, and prints a restock summary.
File Manager	logic2.py	Loads inventory and customers from CSV at startup, and saves them back after every change.
Input Validation	logic1.py	Rejects blank IDs, unknown item IDs, and non-positive or non-numeric quantities, and re-asks where appropriate.
Billing / Receipt	logic1.py	Calculates subtotal, applies the rounded-up tier discount, and prints receipts and restock summaries.

5. Data Structure Design

All of the data structures below are defined in `logic2.py`, which also holds the small functions that change them: `deduct_stock`, `add_stock`, `assign_kiosk`, `release_kiosk`, `add_to_queue`, `take_from_queue`, `add_to_cart` and `record_visit`. `logic1.py` never edits a dictionary or list directly; it always calls one of these.

5.1 Inventory: dictionary keyed by Item ID (hash map)

The inventory is loaded into a dictionary keyed by Item ID. Each key maps to a nested dictionary holding the item's name, quantity and price. This gives $O(1)$ lookup when checking whether an item exists or how much stock it has, instead of scanning every row of the file.

```
inventory = {
    "I001": {"name": "Milk 1L", "quantity": 40, "price": 60},
    "I002": {"name": "Bread", "quantity": 25, "price": 40},
    "I005": {"name": "Cooking Oil 1L", "quantity": 3, "price": 180},
}
```

This dictionary is the in-memory working copy. `inventory.csv` is the persistent copy on disk, kept in sync by loading it at startup and saving the dictionary back after every checkout or restock.

5.2 inventory.csv layout

```
item_id,name,quantity,price
I001,Milk 1L,40,60
I002,Bread,25,40
I005,Cooking Oil 1L,3,180
```

Each row becomes one key-value pair, with `item_id` as the dictionary key.

5.3 Customers: dictionary keyed by Customer ID

Visit history is stored as a dictionary mapping each Customer ID to the number of completed purchases. A customer who is not in the dictionary has zero visits. The visit number for the current checkout is this count plus one, and the tier follows from it.

```
customers = {"C101": 1, "C102": 9}
```

```
customers.csv
customer_id,visits
C101,1
C102,9
```

5.4 Kiosks: dictionary keyed by Kiosk ID

```
kiosks = {"K1": "Busy", "K2": "Free", "K3": "Free", "K4": "Free"}
```

Maps each kiosk to its status. A kiosk is Busy from the moment a customer is assigned until their checkout is completed.

5.5 Active sessions: dictionary keyed by Kiosk ID

While a kiosk is Busy, `active_sessions` remembers who is using it, so Complete Checkout knows whose cart to take. This is what allows several customers to be mid-checkout at the same time.

```
active_sessions = {
    "K1": {"customer_id": "C101", "tier": "Basic", "visit_number": 1}
```

```
}
```

5.6 Customer queue: list used as a FIFO queue

When no kiosk is free, the customer's session (the same small dictionary used above) is appended to a list. When a kiosk is released, `queue.pop(0)` removes the customer at the front, so people are served in the order they arrived.

```
queue = [  
    {"customer_id": "C105", "tier": "Basic", "visit_number": 1}  
]
```

5.7 Cart: list of dictionaries (temporary)

During Complete Checkout, each accepted line is added to a list holding the item name, quantity and line total. It exists only for that one checkout and is never saved.

```
cart = [{"name": "Milk 1L", "quantity": 2, "line_total": 120}]
```

5.8 Summary

Data item	Structure	Lifetime
Inventory	Dictionary of dictionaries, keyed by Item ID	Persistent (inventory.csv)
Customers	Dictionary, Customer ID -> visit count	Persistent (customers.csv)
Kiosks	Dictionary, Kiosk ID -> status	In memory, reset each run
Active sessions	Dictionary, Kiosk ID -> session dictionary	In memory, reset each run
Queue	List used as a FIFO queue	In memory, reset each run
Cart	List of dictionaries	Temporary, one checkout only
Tier discounts	Dictionary: Basic 0, Silver 5, Gold 10	Constant
Low stock threshold	Integer constant (5)	Constant

6. Program Workflow

```
Start Program
|
Load inventory.csv and customers.csv
|
Display Menu (Kiosks + Queue + Inventory)
|
Read User Choice
|
+-----+
| 1 Start Checkout    -> claim kiosk |
|                     or join queue |
| 2 Complete Checkout -> cart, bill, |
|                     release kiosk |
| 3 Process Van Delivery-> restock   |
| 4 View Inventory    |
| 5 Exit              |
+-----+
|
Validate Inputs
|
Perform Operation (update in-memory data)
|
Save CSV files
|
Display Result (receipt / summary / snapshot)
|
Return to Main Menu
|
Exit: save both files, print Goodbye!
```

7. Algorithm Design

Start Checkout

1. Read the Customer ID (ask again if blank).
2. Look up the customer's visit count (0 if unknown); visit number = count + 1.
3. Work out the tier: visit 10 or more is Gold, visit 2 or more is Silver, otherwise Basic.
4. Find the lowest-numbered free kiosk.
5. If one exists, mark it Busy and store the session in active_sessions.
6. If none exists, append the session to the queue and show the customer's position.

Complete Checkout

1. If there are no active sessions, say so and stop.
2. Show the active sessions and read the Kiosk ID to complete; stop if that kiosk has no session.
3. Read cart lines until 'done'. For each line: reject an unknown or blank Item ID; read a positive quantity (ask again if invalid); reject the line if quantity exceeds stock.
4. For an accepted line, deduct the quantity from inventory immediately and add the line to the cart.
5. Save the inventory to inventory.csv.
6. If the cart is empty, print 'No items purchased.' and skip billing.

7. Otherwise compute subtotal, apply the tier discount (rounded up), print the receipt, record the visit in customers and save customers.csv.
8. Mark the kiosk Free and, if the queue is not empty, assign the kiosk to the customer at the front.

Van Restock

1. Read the Van Manifest ID.
2. Read lines until 'done'; reject blank or unknown Item IDs; read a positive quantity (ask again if invalid).
3. Add the quantity to the item's current stock and remember the old and new values.
4. Save the inventory to inventory.csv.
5. Print a restock summary of old and new quantities.

8. Input Design

Input field	Description
Menu choice	1 to 5: Start Checkout, Complete Checkout, Process Van Delivery, View Inventory, Exit.
Customer ID	Entered at Start Checkout. Must not be blank.
Kiosk ID	Entered at Complete Checkout to choose which busy kiosk to finish.
Item ID and Quantity (cart)	One or more lines during Complete Checkout, ended by 'done'.
Van Manifest ID	Reference for a restock delivery. Must not be blank.
Item ID and Quantity Delivered	One or more lines during Van Delivery, ended by 'done'.

9. Output Design

Output	Purpose
Kiosk status	Shows each kiosk's ID and whether it is Free or Busy, with the customer and tier when Busy.
Queue	Lists waiting customers in order, or shows that the queue is empty.
Inventory snapshot	Shows item ID, name, quantity, price, and a LOW STOCK flag where applicable.
Receipt	Shows customer ID, visit number, tier, items, subtotal, discount and total.
Restock summary	Shows each restocked item's quantity before and after the delivery.
Validation messages	Errors for invalid items, insufficient stock, invalid quantities, and invalid menu choices.
Exit message	Displays Goodbye! after saving both files.

10. Validation Design

Validation rule	Action
Invalid menu choice	Display Invalid choice. and show the menu again.
Blank Customer ID, Manifest ID or Item ID	Display Invalid ID. and ask again.
Item ID not in inventory	Reject that line (cart or manifest) with an invalid item message and continue.
Quantity not a positive whole number	Display Invalid quantity value. and ask for the quantity again.
Checkout quantity greater than stock	Reject that line with an insufficient-stock message; inventory unchanged.
All kiosks busy	Add the customer to the queue instead of rejecting them.
No active session at the chosen kiosk	Display a message and return to the menu.
Inventory file missing or malformed	Display a clear error and exit cleanly without a traceback.

11. File Persistence Logic

At startup, `inventory.csv` is read into the inventory dictionary and `customers.csv` into the customers dictionary (an empty dictionary if `customers.csv` does not exist yet). After every completed checkout and every van delivery the inventory is written back to `inventory.csv` in full, overwriting the previous contents. After a successful, non-empty purchase `customers.csv` is rewritten the same way. Each operation completes its update and save before the next one begins, so the files never show a half-finished transaction.

12. Why Checkout Is Split Into Two Steps

A terminal program handles one thing at a time. If checkout were a single step that claimed a kiosk, read the cart and released the kiosk straight away, a kiosk would always be free again before the next menu choice, so the queue could never be triggered by typing. Splitting it into Start Checkout (claim a kiosk or join the queue) and Complete Checkout (cart, billing, release) lets several customers hold kiosks at once. Starting four customers fills all four kiosks, and a fifth joins the queue. Completing a checkout frees a kiosk, which is immediately assigned to the customer at the front of the queue; that customer then still needs their own Complete Checkout.

Alternative considered: running real concurrent checkouts with threads. This would model a real store more closely but adds locking and shared-file complications that are outside the scope of this project.

13. Why the Logic Is Split Into Two Files

Business rules and data handling change for different reasons, so they are kept apart. `logic1.py` holds the rules and calculations: tier calculation, discount rounding, billing, the checkout and restock steps, and the screen output. `logic2.py` holds every data structure, the functions that change them, and the CSV loading and saving. Because `logic1.py` only changes data by calling `logic2.py`, a change to how data is stored (for example, moving from CSV files to a database) would only affect `logic2.py`, and the rules in `logic1.py` would stay the same. Each file is also short enough to explain on its own.

Alternative considered: keeping everything in a single `logic.py`, which was the first version. It worked, but it mixed the rules with the storage code in one file of about 290 lines.

14. Assumptions and Constraints

- Item IDs are unique within the inventory file.
- `inventory.csv` has the columns `item_id`, `name`, `quantity`, `price`; `customers.csv` has `customer_id`, `visits`.
- Quantities and prices are whole numbers.
- Only one instance of the application uses the files at a time.
- A customer holds at most one kiosk session at a time.
- A visit counts toward the loyalty tier only when the customer buys at least one item.
- The van manifest ID is displayed for reference but delivery history is not stored.
- The low stock threshold (5 units) and the number of kiosks (4) are fixed constants.
- The application runs in a terminal and follows a fixed output format.